DHOLE PATIL EDUCATION SOCIETY's
DHOLE PATIL COLLEGE OF ENGINEERING
**Accredited by NAAC with A+ Grade, An ISO 9001:2015 Certified Institute**
**1284, Near Eon IT Park Kharadi, Dhole Patil College Road, Wagholi, Pune-412207**
**Website-www.dpcoepune.edu.in E-mail:** dpcoepune@gmail.com, Phone:020-66059900

**Subject : PPS**

**Notes On Unit-I**

## What is a problem? List down steps in problem solving.

A **problem** is any situation or condition that requires a solution, usually because there is a gap between the current state and the desired outcome. It often involves a challenge or difficulty that needs to be addressed through thought, action, and decision-making.

Here are the general **steps in problem-solving**:

1. **Identify the Problem**
   Clearly define the problem. Understand what the issue is, why it is a problem, and what the desired outcome is.
2. **Analyze the Problem**
   Gather relevant information to understand the causes and effects of the problem. Break the problem down into smaller, manageable parts.
3. **Generate Possible Solutions**
   Brainstorm different ideas and solutions. Consider multiple approaches and think creatively to come up with possible ways to resolve the problem.
4. **Evaluate and Select the Best Solution**
   Assess the pros and cons of each potential solution. Consider factors like feasibility, time, cost, and effectiveness before selecting the best course of action.
5. **Implement the Solution**
   Put the chosen solution into action. This might involve setting up a plan, allocating resources, and carrying out the necessary steps.
6. **Monitor and Review**
   Track the progress of the solution. Monitor the outcomes to ensure that the problem is being resolved. If the solution isn't effective, reassess and adjust as needed.
7. **Reflect on the Process**
   After solving the problem, take time to evaluate the problem-solving process itself. What worked well? What could have been done differently? This helps improve future problem-solving skills.

## What are different types of problems? Explain in brief.

There are two types of problems :

1) **Problems based on algorithmic solutions:**
   Algorithm is nothing but sequence of instructions carried out to solved some problem.
   For solving some problem, series of actions are taken to reached to the solution

For example: problem of making a cup tea can be solved by the sequence of actions.

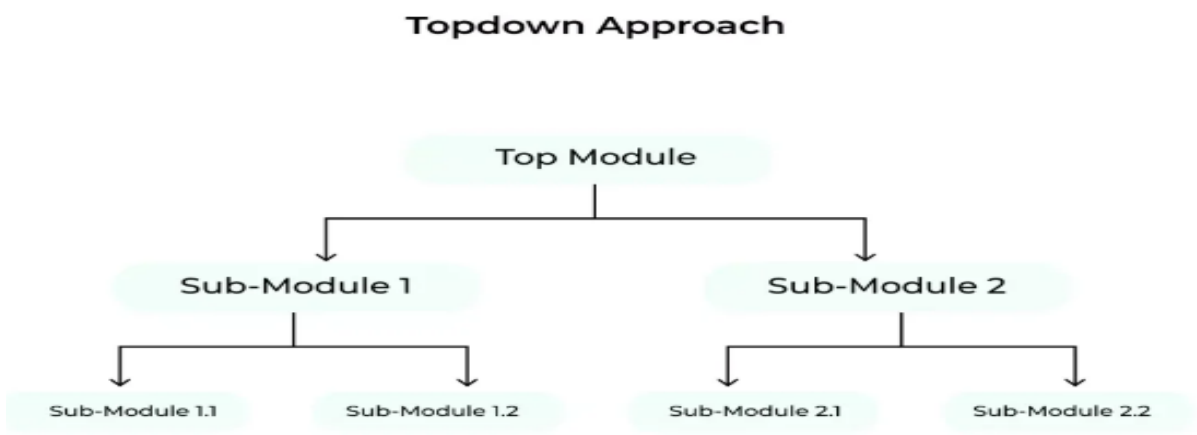
2) **Problems based on heuristic solutions:**
There are some problems that can be solved by just following certain actions. In solving such problems, a critical decision has to be made. Sometimes we need to follow the process of trial and error
The solutions that can not be reached through direct set of steps are called heuristics solutions.
For example – the problem of which stock should I buy is based on heuristic solution as it requires knowledge, experience of trials and error, skills, careful analysis of market and so on.

## What is modularization? Explain top down design approach.

**Modularization:**

Modularization is a design technique in software engineering where a large, complex system is broken down into smaller, manageable, and independent units or modules. Each module focuses on a specific functionality or responsibility, making the system easier to develop, test, debug, maintain, and update.

**Top-down design** is a problem-solving and software design approach where you start by defining the high-level components of the system and then break them down into smaller, more detailed components. It's a methodical approach to software architecture where the system is divided into smaller sub-systems, and each sub-system is further decomposed until the design reaches a level of detail that can be implemented.




**modularization** and the **top-down design approach** are key concepts in software engineering that help in organizing, managing, and implementing complex systems efficiently, making them more maintainable, understandable, and scalable.

## List different program design tools? Explain an algorithm with example

Program design tools help in planning, organizing, and visualizing the structure of a program before implementing it.

1) Algorithm
   An algorithm is a step-by-step procedure or set of rules to perform a specific task or solve a problem. Algorithms are necessary for solving complex problems efficiently and effectively.
2) Flowchart
   Flowcharts are graphical representations of algorithms or processes. They use symbols such as ovals (start/end), rectangles (process), diamonds (decision), and arrows (flow of control) to show the steps in a program.
   Advantages: Easy to understand, visual representation of processes.
3) Pseudocode
   Pseudocode is a way to describe an algorithm using a combination of natural language and programming concepts. It doesn't follow strict syntax rules like a programming language, but it's close enough to help in implementing the solution.
   Advantages: Easy to read, helps clarify logic, language-agnostic.

**What are the characteristics of an algorithm.**

An **algorithm** is a set of well-defined, step-by-step instructions used to solve a problem or perform a task.

Characteristics of an Algorithm:

1. **Finiteness**: The algorithm must have a finite number of steps. It should terminate after a set number of steps.
2. **Definiteness**: Each step of the algorithm must be precisely defined. There should be no ambiguity in the instructions.
3. **Input**: The algorithm should take inputs from the outside world (e.g., user input, data from files, etc.).
4. **Output**: The algorithm should produce an output, which could be a result, transformation, or action.
5. **Effectiveness**: The algorithm's steps should be simple enough to be carried out, typically by a computer, without ambiguity.

**Explain feature of python programming language.**

Python is one of the most popular and widely used programming languages today due to its simplicity, readability, and flexibility.

Here are some of the key features of Python:

1. **Simple and Easy to Learn:** Python's syntax is straightforward and resembles natural language, making it easier for beginners to read and understand code.
2. **Interpreted Language:** Python is an interpreted language, which means that the code is executed line-by-line.

3. **Object-Oriented**: Python supports object-oriented programming (OOP)**,** which means it supports concepts such as classes**,** objects**,** inheritance**,** polymorphism**,** and encapsulation.
4. **High-Level Language**: Python is a high-level language, which means it abstracts away complex machine-level details like memory management.
5. **Large Standard Library**: Python comes with a vast standard library that includes modules for file I/O, regular expressions, networking, web services, and much more.
6. **Cross-Platform**: Python is platform-independent, meaning that code written in Python can run on various operating systems (Windows, macOS, Linux) without modification.

**Explain history of python programming language.**

**1980s**: Python's initial inspiration comes from ABC.
**December 1989**: Guido van Rossum starts Python as a Christmas project.
**February 1991**: Python 0.9.0 released.
**January 1994**: Python 1.0 released.
**October 2000**: Python 2.0 released.
**December 2008**: Python 3.0 released (backward incompatible with Python 2).
**July 2010**: Python 2.7 (final major version of Python 2.x).
**2020s**: Python continues to grow in popularity, with Python 3.x versions dominating the landscape.

**Explain any three Programming paradigms.**

1) Object-Oriented Programming (OOP)
   Object-oriented programming (OOP) is based on the concept of objects — entities that represent both data and behavior.

   Key Concepts:

   - Classes and Objects: A class is a blueprint for creating objects, which are instances of the class.
   - Encapsulation: The concept of bundling data (attributes) and methods (functions) that operate on the data into a single unit (object).
   - Inheritance: The ability to create a new class by inheriting attributes and methods from an existing class.
   - Polymorphism: The ability to use a single interface to represent different underlying forms (data types).
   - Abstraction: Hiding the complexity and showing only essential features.

   **Languages**: Python, Java, C++, Ruby, and C#.

2) **Monolithic programming**
   **Monolithic programming** refers to a software architecture or design approach where an entire application is built as a **single, indivisible unit**. In a **monolithic** system, all the components of the application—such as the user interface, business logic, and data access—are tightly coupled and

share the same codebase. This means that the entire application is developed and deployed as one unified program.

3) **Procedural programming**
   Procedural programming is one of the simplest and most widely used paradigms, especially for small to medium-sized projects. It's based on the idea of dividing a problem into procedures or functions that perform specific tasks.

4) **structured programming**
   In structured programming, we sub-divide the whole program into small modules so that the program becomes easy to understand. The purpose of structured programming is to linearism control flow through a computer program so that the execution sequence follows the sequence in which the code is written.

Explain the following terms: 1) Variable 2) Identifiers

1. **Variable** : A container for storing data that can change during the execution of a program.
   a. **Example** :
      i. x = 5 # 'x' is a variable storing the value 5
      ii. name = "ram" # 'name' is a variable storing the string "Alice"

2. **Identifier :** A name given to variables, functions, and other elements in a program to uniquely identify them.
   Example :
   **Valid identifiers**
   x = 10 # x is a valid identifier
   my_variable = 20 # my_variable is a valid identifier
   sum_result = 30 # sum_result is a valid identifier
   **Invalid identifiers**
   # 1variable = 5 # Invalid because it starts with a digit
   # def = 100 # Invalid because 'def' is a reserved keyword

What is Indentation in Python? Explain with example.

**Indentation** in Python refers to the practice of using whitespace (spaces or tabs) to define the structure of code. It is used to indicate **blocks of code** that belong together, such as those inside loops, functions, conditionals, and classes. In many other programming languages, curly braces {} are used to group code, but in Python, **indentation** is used to indicate the scope and nesting of code blocks.

```
# Example of Indentation in Python
# Function definition
def greet(name):
    if name:
        print("Hello, " + name)        # This line is indented and is part of the 'if' block
```

```
# Calling the function
greet("Ram")
```

Explain different data types in python.

In Python, **data types** refer to the classification of values that determines what operations can be performed on them and how they are stored in memory.

1 **) Numeric Types**

- int (Integer): Represents whole numbers without a decimal point. Can be positive, negative, or zero. ex : x = 5 # An integer
- float (Floating-point): Represents real numbers (i.e., numbers that can have decimal points). ex : a = 3.14 # A floating-point number
- complex: Represents complex numbers with a real and an imaginary part, written as real + imaginary. ex :c = 2 + 3j # A complex number

**2 ) String :** Represents a sequence of characters. Strings are defined using single quotes ' ' or double quotes " ".

ex : my_string = "Hello, Python!" # String using double quotes
another_string = 'Python' # String using single quotes

**3) list**: A mutable, ordered collection of items, which can be of any data type (e.g., integers, strings, objects). Lists are defined using square brackets [ ].
Ex : my_list = [1,2,3,4]

**4) tuple**: An immutable, ordered collection of items. Similar to lists but cannot be modified once created. Tuples are defined using parentheses ( ).
Ex: my_tuple=(1,2,3,4)

**5) dict** (Dictionary): Represents a collection of key-value pairs. Each key is unique, and values can be of any data type. Dictionaries are defined using curly braces { } with key: value pairs.
Ex: my_dict = {"name": "ram", "age": 25, "city": "Pune"}

**Explain input output operations in python with example.**

**Input and Output Operations in Python**

In Python, **input** and **output (I/O)** operations are used to interact with the user and external resources, such as files or the console.

- **Input operations** are used to take input from the user or another source.
  Example:
  name = input("Enter your name: ")

- **Output operations** are used to display information to the user or write to external resources.
  Example:
  name = input("Enter your name: ")

**Write a python program to perform addition of two numbers. Accept the two numbers using keyboard.**

```
# Accepting input from the user
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Performing addition
sum = num1 + num2

# Displaying the result
print("The sum of", num1, "and", num2, "is:", sum)
```